WEEK 3: DYNAMIC PROGRAMMING ON TREES

Advanced Algorithms

ROADMAP

1. **Maximum Independent Set on a Tree**
2. **Minimum Vertex Cover on a Tree**
3. **Maximum Matching on a Tree**
4. **Kőnig and Gallai** — connections between our three problems

Three different problems — *one* algorithmic pattern: root the tree, define a subproblem on subtrees, fill them in **postorder**.

RECAP: HOUSE ROBBER

Remember the house robber problem? Maximise the money you rob, but you cannot rob two *adjacent* houses.



An optimal plan: rob $5 + 10 + 3 = 18$.

This neighborhood has a very simple layout: a **path**.

THE TWO-STATE TRICK, ON A PATH

We can solve house robber in one pass from left to right keeping *two* values per house:

> **STATES** $\mathrm{rob}[i]$ = best haul from houses $0..i$ **robbing** house i;
> $\mathrm{skip}[i]$ = best haul from houses $0..i$ **not robbing** house i.

THE TWO-STATE TRICK, ON A PATH

Base case $\mathrm{rob}[0] = w_0$ and $\mathrm{skip}[0] = 0$.

Update For $i \geq 1$:

$$\mathrm{rob}[i] = w_i + \mathrm{skip}[i-1]$$
$$\mathrm{skip}[i] = \max\left(\mathrm{rob}[i-1],\ \mathrm{skip}[i-1]\right)$$

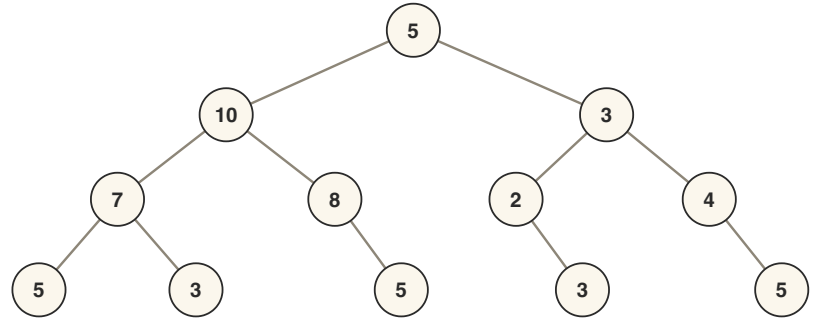
Answer: $\max\left(\mathrm{rob}[n-1], \mathrm{skip}[n-1]\right)$.

Filled left to right — each house looks only at its *neighbor*. Today's theme: the trick survives on a **tree**.

FROM A PATH TO A TREE

What if we want to rob Palm Jumeirah island in Dubai? Its streets branch like a palm frond — the neighborhood is a **tree**.





The island, and our abstraction of it: each vertex is a house with a value, and we cannot rob two houses connected by an edge.

Photo: shre.ae/palm-jumeirah-island-dubai-largest-artifical-archipelago

THE TREE DP RECIPE

Recipe

1. **Root** the tree at any vertex r.
2. Subproblem = the **subtree** T_v rooted at v — that is, v together with everything below it.
3. Keep a few **states** per vertex (e.g. "v used" / "v not used").
4. **Combine**: a vertex's values depend only on its children's values.
5. Fill in **postorder** — children before parents; read off the root.

THE SUBPROBLEM DAG

In weeks 1 and 2 the subproblem DAG was a path or grid, not explicit in the problem. Here the subproblem DAG *is the tree itself*, with edges directed from children to parents.

That recipe is **generic** — we run it three times today, on three different problems. Robbing the island is just the first instance.

WHY THE RECIPE WORKS

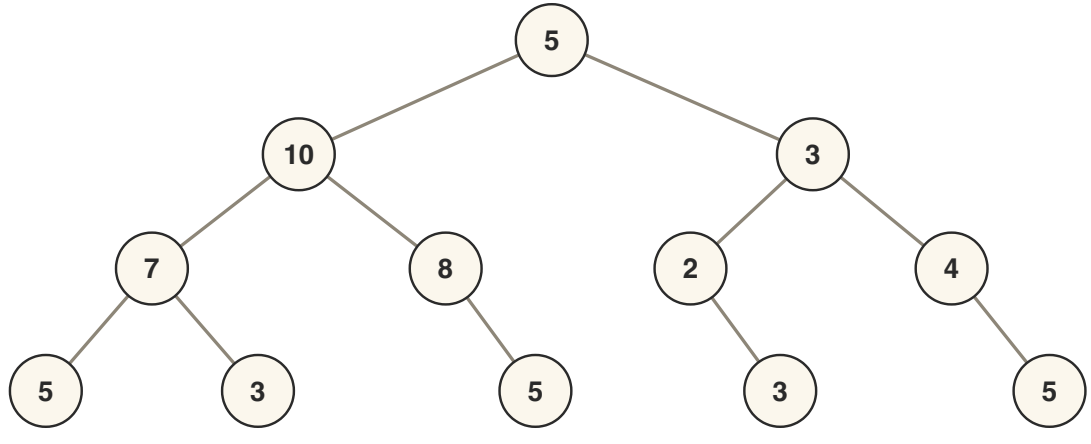
Robbing constrains only **neighbours**, so every constraint sits on an edge. And every edge of T_v either lies inside one child subtree or joins v to a child — **none joins two child subtrees**.

So with v's state fixed the children cannot interfere: the combine step is a sum or a max over children, never a search over combinations.

A subtree touches the rest of the tree only at v. Everything *above* v is outside T_v, and v's state is the whole interface.

ONE TREE, THREE PROBLEMS

This tree is our running example for the whole session (12 vertices, 11 edges; numbers are vertex weights).



Rooted at the top vertex (weight 5).

PART I: MAXIMUM INDEPENDENT SET ON A TREE

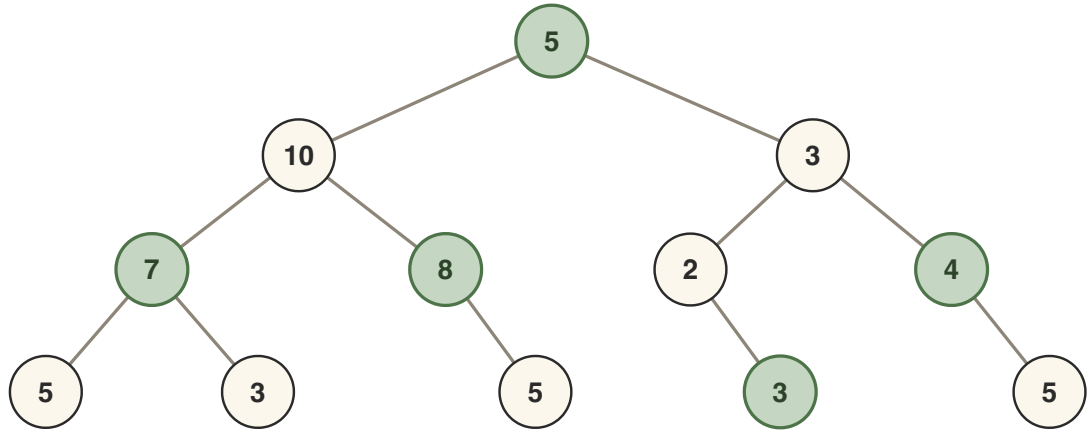
TREE ROBBER

We are robbing the palm-tree island: maximise the total value of robbed houses, but we **cannot rob two houses connected by an edge**.

> **Definition** A set of vertices with no two of them connected by an edge is called an **independent set**.

So the tree robber wants a **maximum weight independent set**: an independent set S maximising $\sum_{v \in S} w_v$, where w_v is the value of house v.

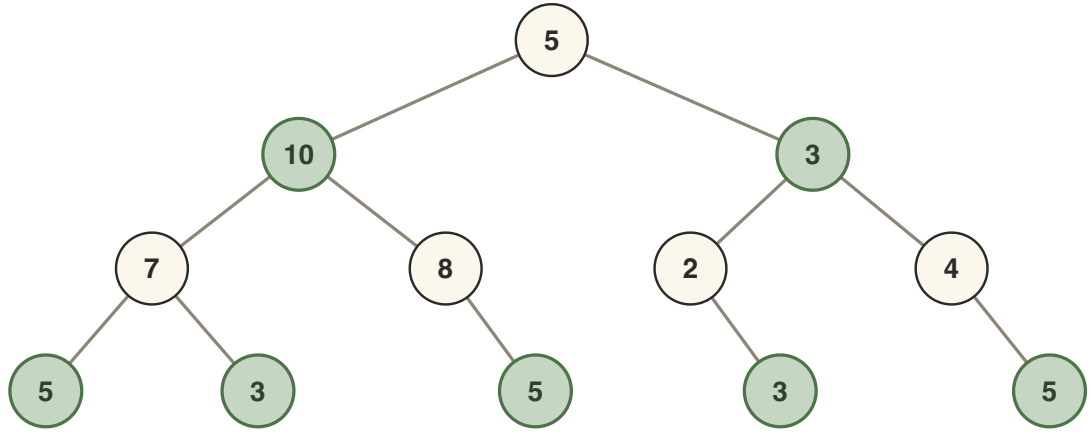
AN INDEPENDENT SET



Green vertices form an independent set of weight $5 + 7 + 8 + 4 + 3 = 27$.

No other vertex can be added without breaking independence. Is 27 the best we can do?

CAREFUL: MAXIMAL $\neq$ MAXIMUM



A better plan: weight $10 + 3 + 5 + 3 + 5 + 3 + 5 = 34$.

CAREFUL A **maximal** independent set (cannot add any vertex) need not be **maximum** (largest weight overall). The weight-27 set was maximal — yet weight 34 is achievable.

MAXIMUM WEIGHT INDEPENDENT SET

Problem Given a tree $T = (V, E)$ with vertex weights $w_v \geq 0$, find the maximum total weight of an independent set in T.

- On general graphs this problem is famously hard — it is NP-complete (Week 11!).
- On trees, dynamic programming solves it in linear time. Let's find the subproblems.

SUBPROBLEMS, TAKE 1

SUBPROBLEM Let $\mathrm{dp}(v)$ be the maximum amount we can rob in the subtree rooted at v.

Key case split: we may or may not rob vertex v.

- If we **rob** v: we cannot rob any of its children — but its grandchildren are fair game.
- If we **don't rob** v: each child w independently contributes its own best, $\mathrm{dp}(w)$.

RECURRENCE, TAKE 1

$$\mathrm{dp}(v) = \max\left(\underbrace{w_v + \sum_{u\,:\,\text{grandchild of } v} \mathrm{dp}(u)}_{\text{rob } v},\ \underbrace{\sum_{w\,:\,\text{child of } v} \mathrm{dp}(w)}_{\text{don't rob } v}\right)$$

Initialisation: for a leaf v, both sums are empty, so $\mathrm{dp}(v) = w_v$.

Answer: $\mathrm{dp}(\mathrm{root})$.

This is correct — and still $O(n)$ overall, since each vertex is a child of one vertex and a grandchild of at most one vertex.

TAKE 1 WORKS, BUT...

- The recurrence reaches *two levels down*: to fill $\mathrm{dp}(v)$ we must iterate over children *of children*.
- That makes the code fiddlier and breaks the clean pattern "a vertex only talks to its children".
- Just like house robber, there is an alternative formulation which is **easier to implement**: put more information *into the state* instead of reaching deeper into the tree.

> **MORAL** When a recurrence wants to peek past its immediate neighbors, enrich the state instead.

ALTERNATIVE DEFINITION: TWO STATES

STATES Let $\mathrm{dp}_1(v)$ be the maximum profit from robbing the subtree rooted at v **when we rob** v.

Let $\mathrm{dp}_2(v)$ be the maximum profit from robbing the subtree rooted at v **when we do not rob** v.

This is exactly $\mathrm{rob}[i]$ / $\mathrm{skip}[i]$ from the path version — the state records the decision at v, so the parent never needs to look past its children.

MIS: THE UPDATE

If we rob v, no child may be robbed:

$$\mathrm{dp}_1(v) \;=\; w_v + \sum_{u\,:\,\text{child of } v} \mathrm{dp}_2(u)$$

If we do not rob v, each child is free to make its own best choice:

$$\mathrm{dp}_2(v) \;=\; \sum_{u\,:\,\text{child of } v} \max\left(\mathrm{dp}_1(u),\, \mathrm{dp}_2(u)\right)$$

MIS: THE UPDATE

Asymmetry Skipping v does *not* force us to rob the children — each child independently takes whichever of $\mathrm{dp}_1(u)$, $\mathrm{dp}_2(u)$ is better.

Robbing v leaves the children no choice: only $\mathrm{dp}_2(u)$ is allowed.

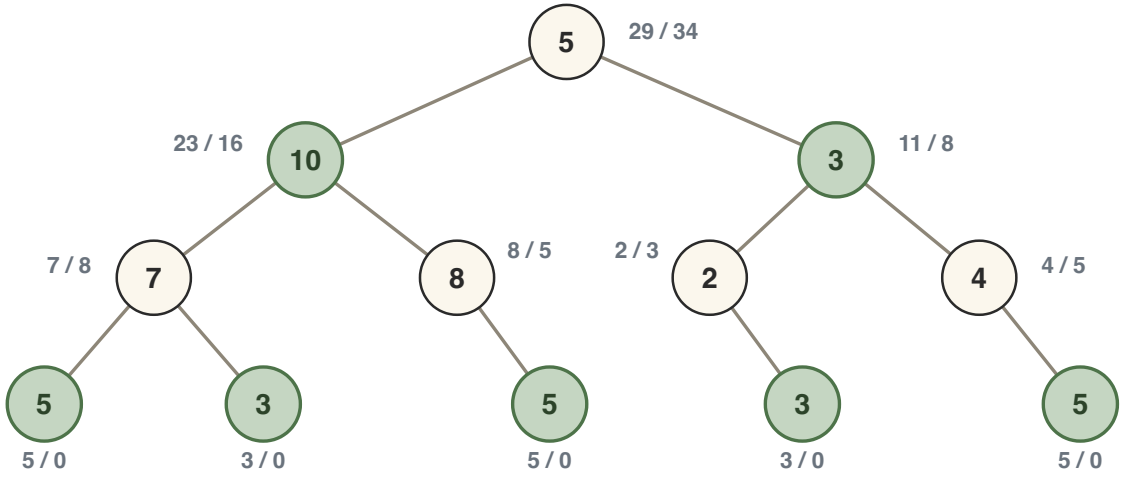
MIS: INITIALISATION AND ANSWER

INITIALISATION If v is a leaf: $\mathrm{dp}_1(v) = w_v$ and $\mathrm{dp}_2(v) = 0$.

ANSWER $\max\left(\mathrm{dp}_1(\mathrm{root}),\ \mathrm{dp}_2(\mathrm{root})\right)$

How do we order the computation? When we process a vertex v, we want to have already processed its children… **postorder traversal!**

MIS: WORKED EXAMPLE



Beside each vertex: $\mathrm{dp}_1(v)$ / $\mathrm{dp}_2(v)$. Green = the optimal set traced back from the root.

At the root: $\mathrm{dp}_1 = 5 + 16 + 8 = 29$ and $\mathrm{dp}_2 = \max(23, 16) + \max(11, 8) = 34$. Answer: $\max(29, 34) = 34$ — don't rob the root.

MIS: IMPLEMENTATION

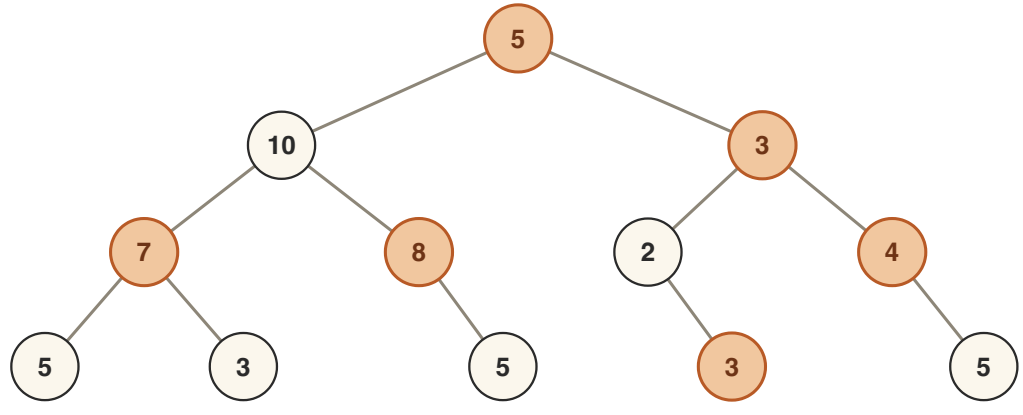
```
// dp1[v]: best in subtree of v if we rob v
// dp2[v]: best in subtree of v if we do not rob v
void dfs(int v, int parent) {
    dp1[v] = w[v];                          // leaf initialisation…
    dp2[v] = 0;
    for (int u : adj[v]) {
        if (u == parent) continue;
        dfs(u, v);                          // postorder: children first
        dp1[v] += dp2[u];                   // …updated as children return
        dp2[v] += max(dp1[u], dp2[u]);
    }
}
// answer: max(dp1[root], dp2[root])
```

- Store the tree as adjacency lists; passing `parent` stops the DFS from walking back up.
- Time $O(n)$; recursion depth can hit n on a path — for huge inputs use an explicit stack.

PART II: MINIMUM VERTEX COVER ON A TREE

VERTEX COVERS

DEFINITION A **vertex cover** is a subset of vertices S such that every edge has an endpoint in S.



Orange vertices cover all 11 edges: a vertex cover of weight $5 + 3 + 7 + 8 + 4 + 3 = 30$.

Check for yourself: every edge touches an orange endpoint. Is 30 the *minimum* weight possible?

MINIMUM WEIGHT VERTEX COVER

Problem Given a tree $T = (V, E)$ with vertex weights $w_v \geq 0$, find a vertex cover of minimum total weight.

Think of it as placing guards: every street (edge) must be watched by a guard at one of its ends, and guard v costs w_v.

MINIMUM WEIGHT VERTEX COVER

Same recipe: root the tree, and for each vertex remember the one bit the parent needs — *is v in the cover or not?*

STATES $\mathrm{in}(v)$ = min weight to cover all edges of T_v with v **in** the cover;
$\mathrm{out}(v)$ = min weight to cover all edges of T_v with v **not in** the cover.

VC: THE RECURRENCE

If v is in the cover, every child edge is covered — each child chooses freely:

$$\text{in}(v) \;=\; w_v + \sum_{u\,:\,\text{child of } v} \min\big(\text{in}(u),\, \text{out}(u)\big)$$

If v is *not* in the cover, each edge (v, u) needs its other end — every child is **forced in**:

$$\text{out}(v) \;=\; \sum_{u\,:\,\text{child of } v} \text{in}(u)$$

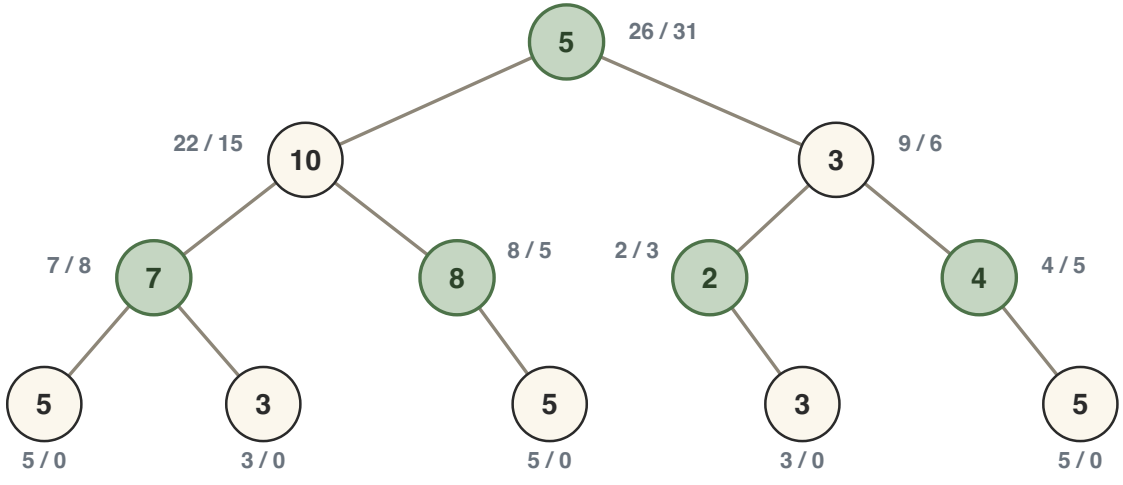
VC: INITIALISATION AND ANSWER

INITIALISATION If v is a leaf (no edges inside T_v):
$\mathrm{in}(v) = w_v$ and $\mathrm{out}(v) = 0$.

ANSWER $\min\big(\mathrm{in}(\mathrm{root}),\ \mathrm{out}(\mathrm{root})\big)$

Fill in postorder, exactly as before. To output the cover itself, trace back from the root: if v is out, all its children are in; if v is in, each child picks its cheaper option.

VC: WORKED EXAMPLE



Beside each vertex: $\text{in}(v)$ / $\text{out}(v)$. Green = the minimum cover, traced back from the root.

Root: $\text{in} = 5 + \min(22, 15) + \min(9, 6) = 26,\ \text{out} = 22 + 9 = 31.$

Minimum weight 26 — beating our earlier cover of weight 30.

VC: IMPLEMENTATION

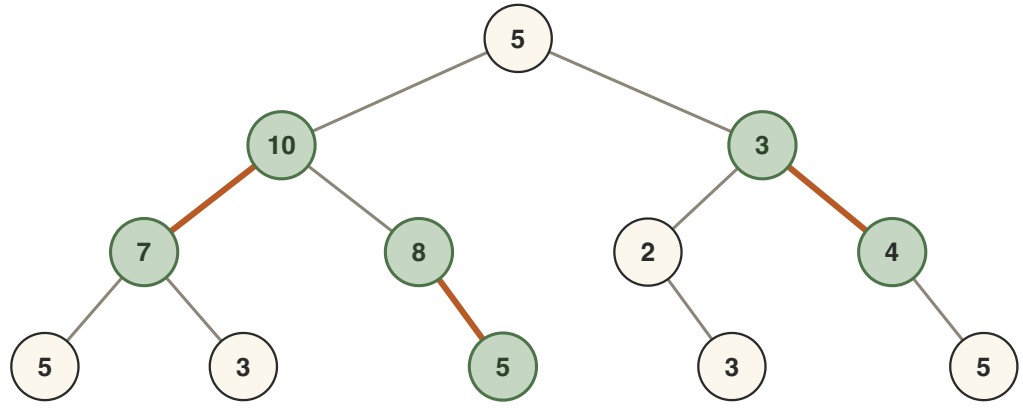
```
// in_[v]: min cover of subtree of v, v in the cover
// out[v]: min cover of subtree of v, v not in the cover
void dfs(int v, int parent) {
    in_[v] = w[v];                          // leaf values…
    out[v] = 0;
    for (int u : adj[v]) {
        if (u == parent) continue;
        dfs(u, v);                          // postorder
        in_[v] += min(in_[u], out[u]);      // child free to choose
        out[v] += in_[u];                   // child forced into cover
    }
}
// answer: min(in_[root], out[root])
```

Structurally identical to the MIS code — swap $\max \leftrightarrow \min$ and flip which state constrains the children. Time $O(n)$.

PART III: MAXIMUM MATCHING ON A TREE

MATCHINGS

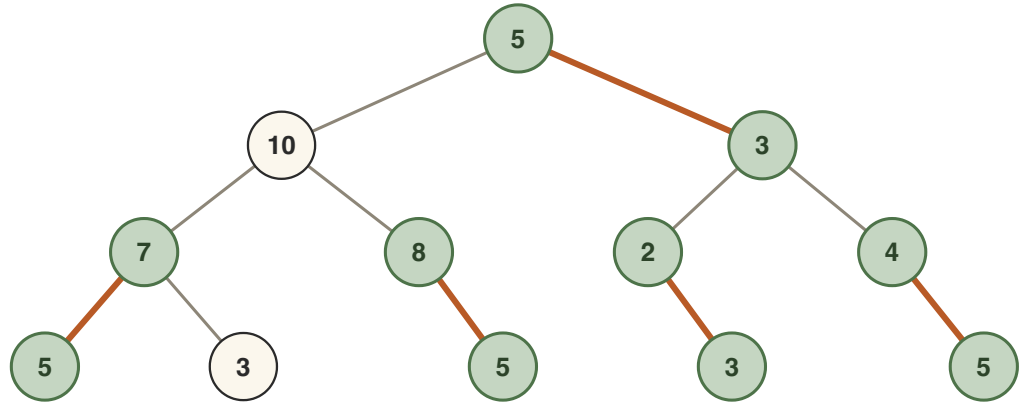
Definition A **matching** is a set of edges such that no two edges in the set share an endpoint.



Three edges, no shared endpoints: a matching of size 3. Matched vertices in green.

MAXIMUM MATCHING ON A TREE

PROBLEM Given a tree $T = (V, E)$, find the size of a largest matching in T.

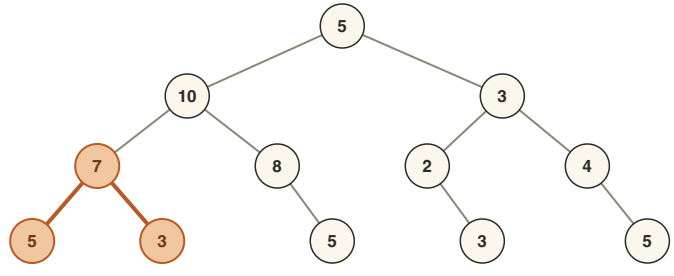


A matching of size 5 in our tree. (Vertex weights play no role in this problem.)

Can we do better than 5?

HOW LARGE CAN A MATCHING BE?

Every matched edge uses 2 vertices, so with $n = 12$ vertices any matching has size at most $\lfloor n/2 \rfloor = 6$.



Size 6 means a **perfect matching** — every vertex used.

But the two highlighted leaves share their only neighbor: at most one can be matched. So 6 is impossible, and 5 is maximum.

SUBPROBLEMS FOR MATCHING

Root the tree. For each vertex v, consider the best matching inside the subtree T_v — but we must remember whether v itself is still available to be matched upward.

> **STATES** $\mathrm{free}(v)$ = size of a largest matching in T_v in which v is **unmatched**;
> $\mathrm{best}(v)$ = size of a largest matching in T_v (no constraint on v).

Two states per vertex, exactly as in the tree robber — but this time the decision recorded at v is whether v is still available to its parent, not whether we robbed it.

MATCHING: THE RECURRENCE

If v stays unmatched, each child subtree contributes its own best matching, independently:

$$\text{free}(v) \;=\; \sum_{u \,:\, \text{child of } v} \text{best}(u)$$

If v is matched, it is matched to exactly one child u; that child must have been free below:

$$\text{take}(v) \;=\; \max_{u \,:\, \text{child of } v} \left(1 + \text{free}(u) + \sum_{u' \neq u} \text{best}(u')\right)$$

MATCHING: THE RECURRENCE

Finally, take the better of the two states:

$$\mathrm{best}(v) \;=\; \max\big(\mathrm{free}(v),\, \mathrm{take}(v)\big)$$

Implementation trick:
$\mathrm{take}(v) = \mathrm{free}(v) + \max_u \big(1 + \mathrm{free}(u) - \mathrm{best}(u)\big)$ — one pass over the children, no "all but one" re-summing.

MATCHING: INITIALISATION AND ANSWER

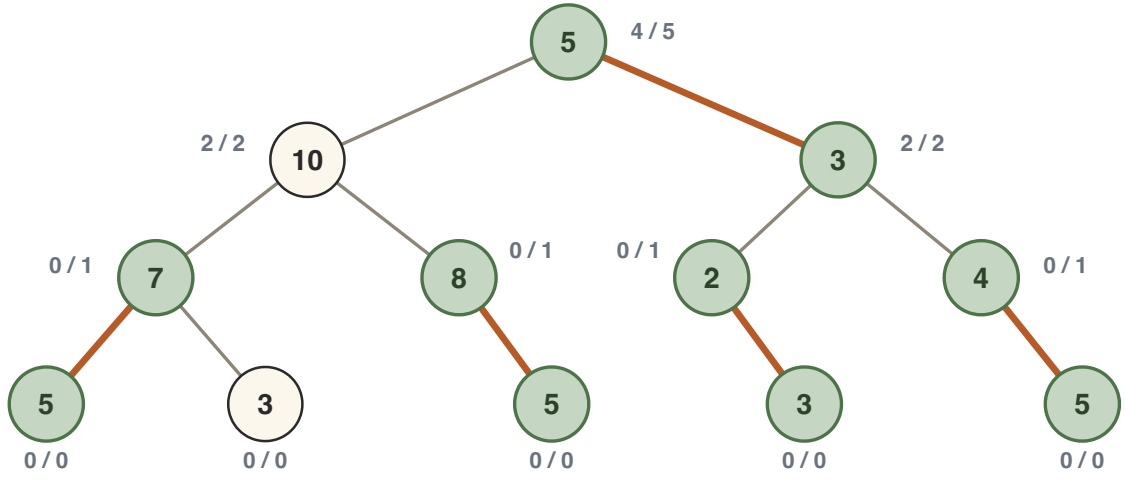
Where does the recursion bottom out? At the leaves.

INITIALISATION If v is a leaf: $\mathrm{free}(v) = 0$, and $\mathrm{take}(v)$ is impossible (no children), so $\mathrm{best}(v) = 0$.

ANSWER $\mathrm{best}(\mathrm{root})$ — the root of the whole tree has no parent, so no constraint applies.

To compute $\mathrm{free}(v)$ and $\mathrm{best}(v)$ we need the values of *all children first* — postorder again.

MATCHING: WORKED EXAMPLE



Beside each vertex: $\mathrm{free}(v)$ / $\mathrm{best}(v)$, filled bottom-up. One maximum matching in orange.

At the root: $\mathrm{free} = 2 + 2 = 4$; $\mathrm{take} = 1 + 2 + 2 = 5$ (match the root to either child); $\mathrm{best}(\mathrm{root}) = 5$.

MATCHING: IMPLEMENTATION

```
// returns {free, best} for the subtree rooted at v
pair<int,int> dfs(int v, int parent) {
    int fre = 0, gain = INT_MIN;            // gain: best bonus for matching v
    for (int u : adj[v]) {
        if (u == parent) continue;
        auto [f, b] = dfs(u, v);            // children first!
        fre  += b;                          // v stays unmatched
        gain  = max(gain, 1 + f - b);       // or: match v with u
    }
    int best = (gain == INT_MIN) ? fre      // leaf: no children
                                 : max(fre, fre + gain);
    return {fre, best};
}
// answer: dfs(root, -1).second
```

One-pass trick: matching v to u shifts the full sum `fre` by $1 + f - b$ — so `fre + gain` is the best "all children but one" sum.

Every vertex and edge is touched $O(1)$ times: $O(n)$ total.

MATCHING: REMARKS

- The same two-state DP also solves *maximum weight* matching on a tree — replace the $+1$ with the edge weight.
- On general graphs, maximum matching is much harder (Edmonds' blossom algorithm); on *bipartite* graphs we will solve it with max-flow in Week 8.
- Preview of Week 4: on trees a simple **greedy** rule — repeatedly match a leaf to its parent — is also optimal.

PART IV: KŐNIG AND GALLAI

Connections between our three problems

MATCHINGS BOUND COVERS

Write $\nu(G)$ for the maximum matching size and $\tau(G)$ for the minimum vertex cover size.

> **Weak duality** $\nu(G) \leq \tau(G)$ in *every* graph.

Proof. A vertex cover C must contain at least one endpoint from every edge of a matching M. The edges of M are disjoint, so distinct edges contribute distinct vertices to C: $|M| \leq |C|$. ■

So a matching is a **certificate**: exhibit one of size k and every cover is forced to have at least k vertices.

KŐNIG'S THEOREM

Theorem (Kőnig, 1931) If G is **bipartite**, then $\nu(G) = \tau(G)$.

Trees are bipartite — 2-colour by depth. On our tree with unit weights: maximum matching 5, minimum cover 5.

Bipartite is essential: a triangle has $\nu = 1$ but $\tau = 2$.

The proof is **constructive**: it turns a maximum matching into a cover of exactly the same size.

A TIGHT COVER HAS NO FREEDOM

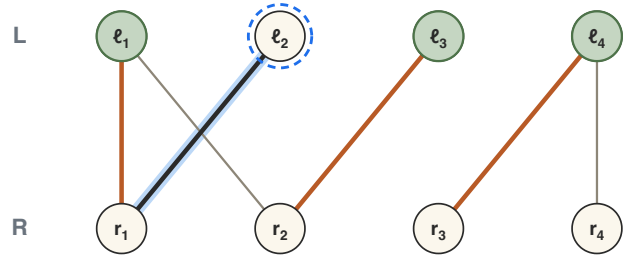
Aim for a **tight** cover — one with $|C| = |M|$, the smallest weak duality allows. What must such a C look like?

- The $|M|$ matching edges are disjoint and C must hit every one — that alone spends the whole budget.
- So C takes **exactly one endpoint of each matching edge** — never both — and **no other vertex at all**.
- In particular **no unmatched vertex is in C.**

NO DESIGN FREEDOM Building C is one binary choice per matching edge: **left endpoint or right?**

WHICH ENDPOINT? TRY THE LEFT

Simplest guess: take the **left** endpoint of every matching edge. Exactly $|M|$ vertices, on budget. What does it miss?

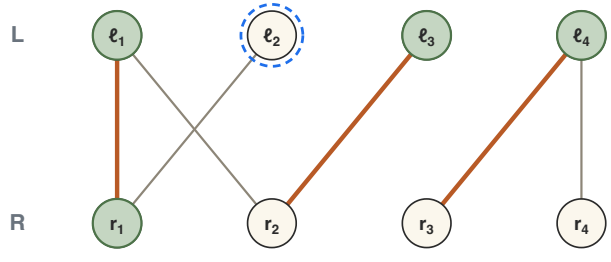


Green = chosen, orange = matching. Every edge is covered except the blue-highlighted one, $\{\ell_2, r_1\}$.

THE WHOLE DIFFICULTY The uncovered edges are exactly those leaving an **unmatched** left vertex. They must be covered from the **right**.

FORCED MOVE: THE SEED

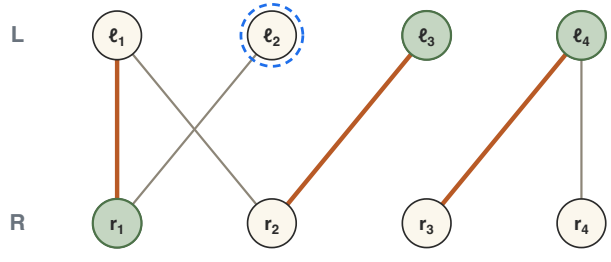
ℓ_2 is unmatched, so $\ell_2 \notin C$. Its neighbour r_1 is therefore **forced in**.



Every edge is covered now — but $|C| = 4 > 3 = |M|$.

This forces ℓ_1 **out** of the cover.

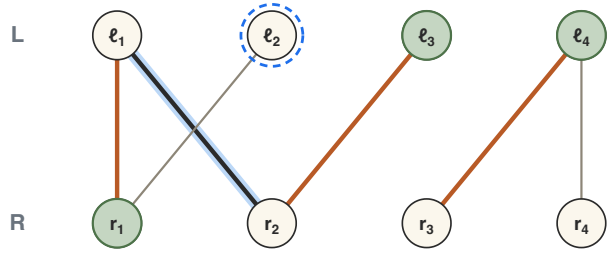
FORCED MOVE: ℓ_1 DROPS OUT



r_1 swaps in for ℓ_1: back to $|C| = 3$, on budget.

FORCED MOVE: THE DAMAGE

ℓ_1 is out of C — but ℓ_1 had a *second* edge, and nothing covers it any more.

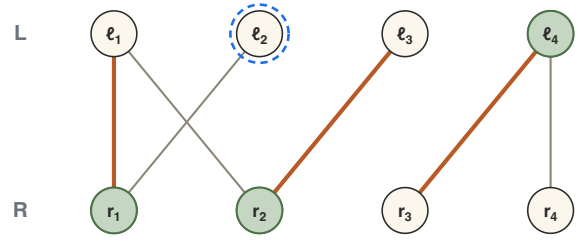


$\{\ell_1, r_2\}$ is now the uncovered edge.

SAME SITUATION, ONE STEP ALONG An uncovered edge whose left end is out of C — so its **right** end is forced in, and we repeat.

FORCED MOVE: THE CHAIN

ℓ_1 is out, so $\{\ell_1, r_2\}$ must be covered by r_2 — which pushes ℓ_3 out.

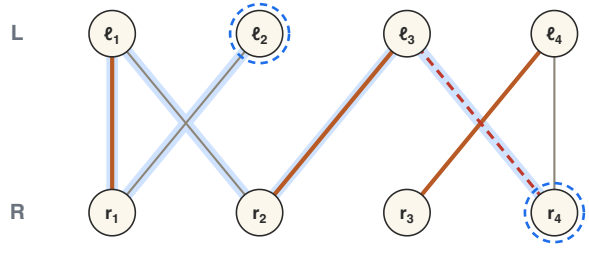


$C = \{r_1, r_2, \ell_4\}$ — one endpoint per matching edge.

- ℓ_3's only edge is already covered by r_2, so nothing new is forced: the chain **stops**.
- Every edge now has an endpoint in C: **C is a vertex cover**.
- $|C| = 3 = |M|$, and *every* choice was forced.

WHAT COULD BREAK THE CHAIN?

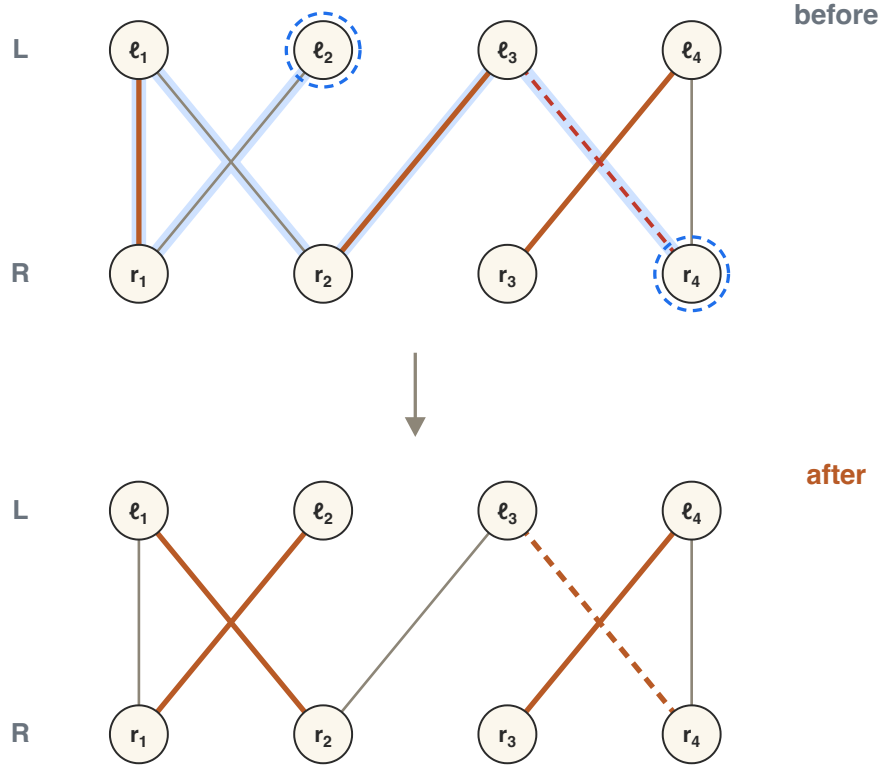
Every forced step needs the vertex it pulls in on the right to be *matched*. Suppose ℓ_3 also had an edge to the unmatched r_4: then r_4 is forced in, sits on no matching edge, and the budget breaks.



Both circled ends are unmatched, and the highlighted walk between them alternates **off**, on, **off**, on, **off** the matching.

SWAP ALONG THE WALK

Swap the matching and non-matching edges along the walk.



Still a matching — and the walk had one more off-matching edge than matching edges, so we gain exactly one: $|M| = 3 \to 4$.

AUGMENTING PATHS

Definition An **augmenting path** starts and ends at *unmatched* vertices and alternates off, on, . . ., off the matching.

- Swapping along one always gives a matching with **one more** edge — so a maximum M admits *no* augmenting path.
- The forced chain walks exactly such an alternating path out of the unmatched ℓ_2.
- If it ever reached an *unmatched* right vertex, that walk would be augmenting — contradicting M maximum.

The chain never breaks Every vertex forced in on the right is matched, so it **swaps in** for its matched neighbour instead of adding to C: $|C| = |M|$.

WHY THE PROCESS STOPS

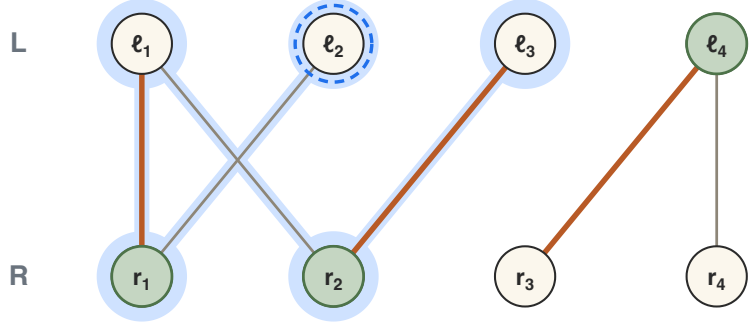
Every forced move does the same thing to *one* matching edge: its chosen endpoint moves from the left end to the right end.

- Moves only ever **add** right vertices and **drop** left ones, so no matching edge ever flips back.
- There are $|M|$ matching edges, so at most $|M|$ moves: the process **stops**.
- And it can only stop with *no* uncovered edge — an uncovered edge has its left end out of C and its right end matched, which forces one more move.

KŐNIG, CONSTRUCTIVELY The process stops, and the C it leaves is a vertex cover with $|C| = |M|$ — exactly what the theorem asks for.

KŐNIG: EXPLICIT CONSTRUCTION

Same argument, stated **explicitly**: instead of running the swaps, we write the cover down. Trace the alternating paths from *every* unmatched left vertex and name three sets:

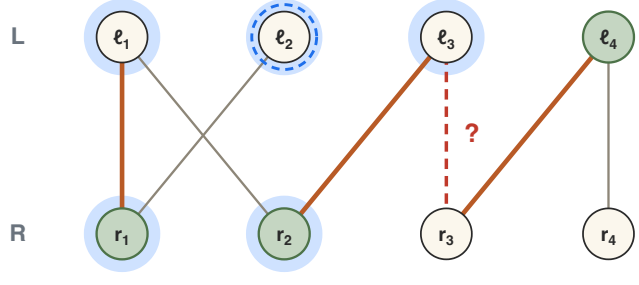


- U = the **seeds**: the M-unmatched vertices of L (here just ℓ_2).
- Z = everything reachable from U by a path that **alternates**: non-matching, matching, non-matching, … (so $U \subseteq Z$).
- $C = (L \setminus Z) \cup (R \cap Z)$: the left vertices Z *missed* plus the right ones it *reached* — exactly the matching edges the swaps flipped.

KŐNIG: WHY C COVERS EVERY EDGE

U = unmatched in L, Z = alternating-reachable from U, C = $(L \setminus Z) \cup (R \cap Z)$

Outside C: left vertices *in* Z, right vertices *outside* Z. So an uncovered edge $\{\ell, r\}$ would have to **escape**: $\ell \in Z, r \notin Z$.



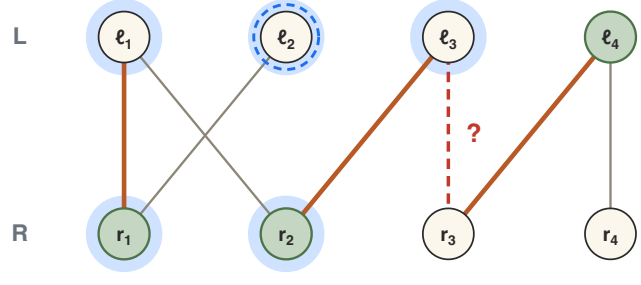
The dashed edge is hypothetical: could some edge leave Z like this?

CASE 1 $\{\ell, r\} \notin M$: the path that put ℓ into Z ended on ℓ's matching edge (or $\ell \in U$) — so it extends along $\{\ell, r\}$, and $r \in Z$. No escape.

KŐNIG: WHY C COVERS EVERY EDGE

U = unmatched in L, Z = alternating-reachable from U, C = $(L \setminus Z) \cup (R \cap Z)$

Still hunting an uncovered edge $\{\ell, r\}$ — one that **escapes**: $\ell \in Z, r \notin Z$. One case left:



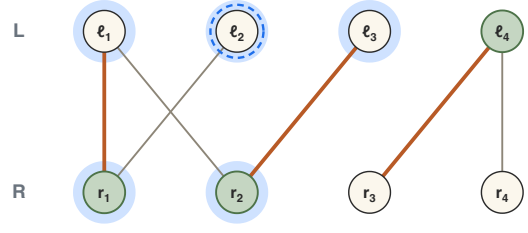
Could the dashed edge be in M? But ℓ_3 entered Z via its matching edge from r_2.

CASE 2 $\{\ell, r\} \in M$: ℓ is matched, so $\ell \notin U$ — the path *entered* ℓ through $\{\ell, r\}$ itself, coming from r. So $r \in Z$ already. No escape.

Both cases force $r \in R \cap Z \subseteq C$: no edge escapes, C is a cover. ■

KŐNIG: WHY |C| = |M|

U = unmatched in L, Z = alternating-reachable from U, $C = (L \setminus Z) \cup (R \cap Z)$



Each vertex of C sits on its own matching edge: $|C| = 3 = |M|$.

- Every vertex of C is matched: an unmatched left vertex would be in $U \subseteq Z$, hence outside C…
- …and an unmatched $r \in R \cap Z$ would end an **augmenting path**: swap its matching and non-matching edges and M grows — but M is *maximum*. (The only place maximality is used!)
- The matching edges are *distinct*: an M-edge with right end in Z has left end in Z too. So $|C| \leq |M|$.
- By weak duality $|M| \leq |C|$, thus $|C| = |M|$. ■

GALLAI'S THEOREM

A SHORTCUT: COMPLEMENTS

OBSERVATION S is an independent set $\iff$ $V \setminus S$ is a vertex cover.

Proof. ($\Rightarrow$) If S is independent, no edge has both endpoints in S, so every edge has an endpoint in $V \setminus S$: a cover.

($\Leftarrow$) If $V \setminus S$ is a cover, no edge has both endpoints outside it — i.e. no edge has both endpoints in S: independent. ■

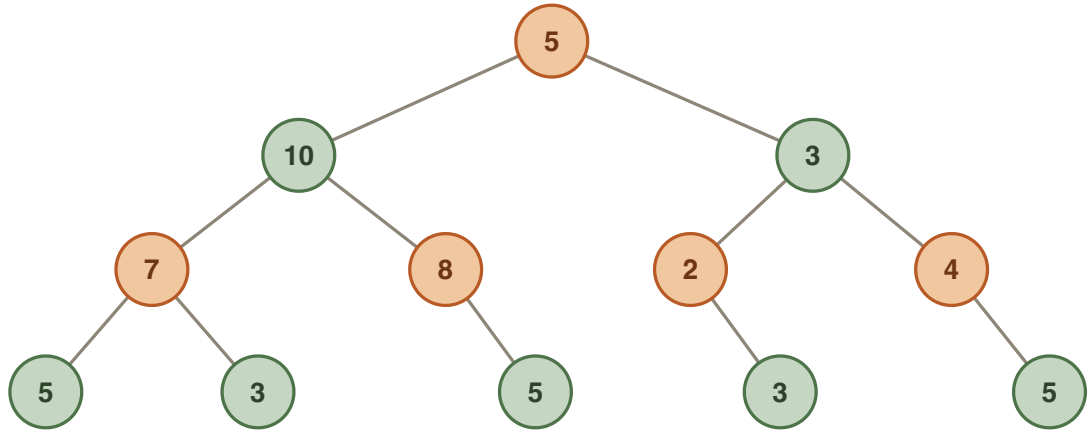
A SHORTCUT: COMPLEMENTS

Consequence

$$\min_{\text{cover } C} w(C) \;=\; w(V) \;-\; \max_{\text{ind. set } S} w(S)$$

A **max-weight MIS** solver instantly gives a **min-weight VC** solver: take the complement!

COMPLEMENTS ON OUR TREE



Green = maximum independent set (weight 34). Orange = its complement, the minimum vertex cover (weight 26).

Total weight of the tree: 60. And indeed $34 + 26 = 60$ — the two answers are two halves of the same split.

GALLAI IDENTITIES

Theorem (Gallai, 1959) In any graph G on n vertices:

$$\alpha(G) + \tau(G) = n$$

where α = size of a maximum independent set and τ = size of a minimum vertex cover. With weights:
$w(S^*) + w(C^*) = w(V)$.

GALLAI IDENTITIES

DEFINITION An **edge cover** of G is a set of edges $F \subseteq E$ such that every vertex of G is an endpoint of some edge in F.

A companion identity relates matchings to edge covers: $\nu(G) + \rho(G) = n$ for graphs with no isolated vertices (ν = max matching, ρ = min edge cover).

THE VERTEX COVER ARC

Vertex cover is where the difficulty shows as graphs get wilder:

Week	Setting	Vertex cover status
W3	trees	linear-time DP (today)
W10	bipartite	polynomial, by Kőnig
W11	general	NP-complete
W12	general	2-approximation

Matching and vertex cover meet again as an LP-duality capstone in Week 10.

SUMMARY: THREE PROBLEMS, ONE RECIPE

Problem	States at v	Parent constraint	Answer
Max weight IS	$\mathrm{dp}_1(v)$, $\mathrm{dp}_2(v)$	take $v \Rightarrow$ children out	$\max(\mathrm{dp}_1(r), \mathrm{dp}_2(r)) = 34$
Min weight VC	$\mathrm{in}(v)$, $\mathrm{out}(v)$	skip $v \Rightarrow$ children in	$\min(\mathrm{in}(r), \mathrm{out}(r)) = 26$
Max matching	$\mathrm{free}(v)$, $\mathrm{best}(v)$	match $v \Rightarrow$ needs a free child	$\mathrm{best}(r) = 5$

Takeaway Root the tree · state = (subtree, decision at v) · initialise at leaves · combine children · **one postorder pass**, $O(n)$ total.